

Artificial Intelligence

AI-2002

Lab 09

Dynamic Bayesian
Networks, Markov's Model

National University of Computer & Emerging Sciences –
NUCES – Karachi



National University of Computer & Emerging Sciences -
NUCES - Karachi
FAST School of Computing

Course Code: AI-2002 Artificial Intelligence Lab

1. Objective 3 2. Bayesian Network 3 2.1 Why use Bayesian Network 3 2.2
Components of a Bayesian Network 3 2.3 Elements of a bayesian Network 4
3. Genie Software Installation 4 4. Burglary Alarm Problem using Bayesian
Network 9 5. Markov Model 12 6. Lab Tasks 14



National University of Computer & Emerging Sciences -
NUCES - Karachi
FAST School of Computing

1. Objective

1. Introduction to Bayesian Network and Dynamic Bayesian Networks (DBN).
2. Explore how to use python to solve BN, DBN problems and then apply these skills to a couple of challenge problems.
3. Introduction to different Libraries of python.

2. Bayesian Network

A Bayesian Network is a probabilistic graphical model that represents a set of variables and their conditional dependencies through a **Directed Acyclic Graph (DAG)**. It is also referred to as a Bayes network, belief network, decision network, or Bayesian model. Bayesian networks are built from probability distributions and leverage probability theory for tasks such as prediction, anomaly detection, diagnostics, and decision-making under uncertainty

2.1 Why Use Bayesian Network

Real-world systems often involve uncertainty. Bayesian networks help model the probabilistic relationships between multiple events. They are useful in: •
Prediction

- Anomaly detection
- Diagnostic systems
- Automated reasoning and insight generation
- Time series prediction
- Decision-making under uncertainty
- Components of a Bayesian Network

2.2 Components of a Bayesian Network

A Bayesian Network consists of two main components:

1. **Directed Acyclic Graph (DAG):** Represents the structure of dependencies between variables.
2. **Conditional Probability Tables (CPTs):** Specify the quantitative strength of those dependencies.



2.3 Elements of a Bayesian Network

- **Nodes (or Vertices):** Represent random variables in the domain being modeled.

These can be:

- Observable variables (e.g., symptoms)
- Latent or hidden variables (e.g., disease presence)

- **Edges:** Directed links between nodes that represent probabilistic dependencies.

An edge from node $A \rightarrow B$ implies that B is conditionally dependent on A . ●

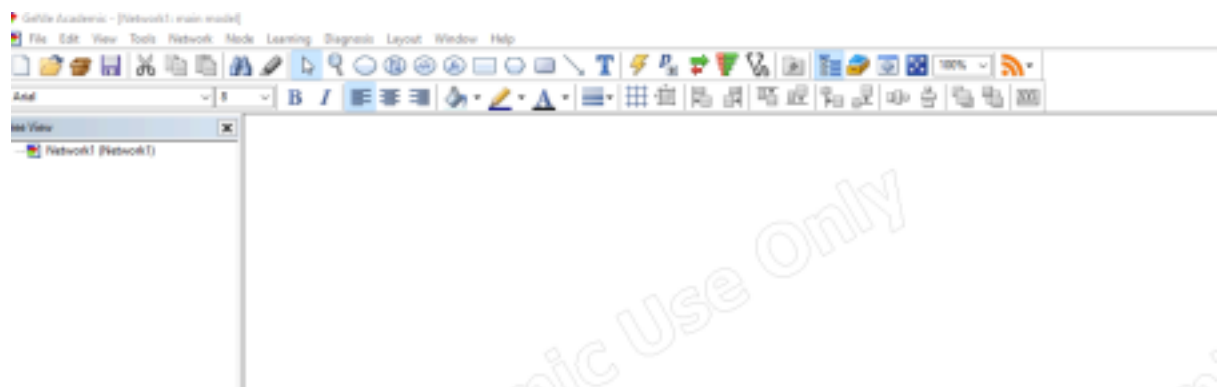
Conditional Probability Tables (CPTs): Each non-root node is associated with a CPT that defines the probability distribution of that variable given its parent nodes.

- **Parent Nodes:** Nodes that have edges pointing to a specific node. A node's value depends on its parent nodes.
- **Root Nodes:** Nodes with no incoming edges. These are independent variables or inputs to the system.
- **Leaf Nodes:** Nodes with no outgoing edges. They often represent outputs or final observed variables.
- **Network Structure:** The overall connection pattern among the nodes. This structure determines conditional independencies and supports efficient probabilistic inference.

3. Genie Software

<https://download.bayesfusion.com/files.html?category=Academia>

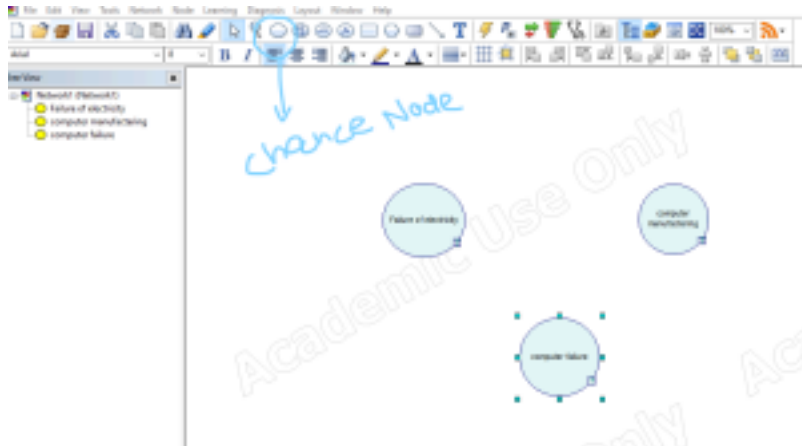
- Open this link and download the setup following the steps to install the software. After installation it will have this kind of interface



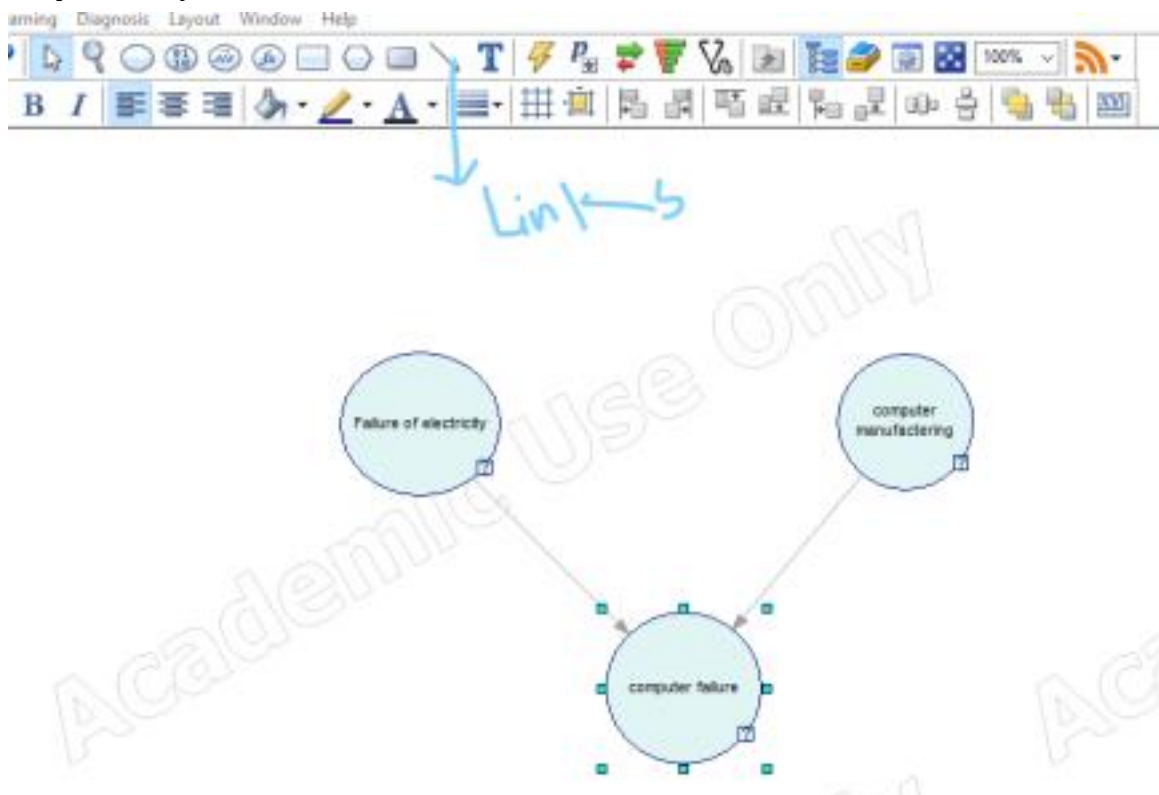


National University of Computer & Emerging Sciences -
NUCES - Karachi
FAST School of Computing

- There are multiple tabs to draw a network. Let's follow the steps to draw a network with 3 nodes.



- This is a chance node. Here we have drawn 3 nodes. the next step is to set the links between these. The arrows are in light gray color which suggests that you have not defined the probability tables .



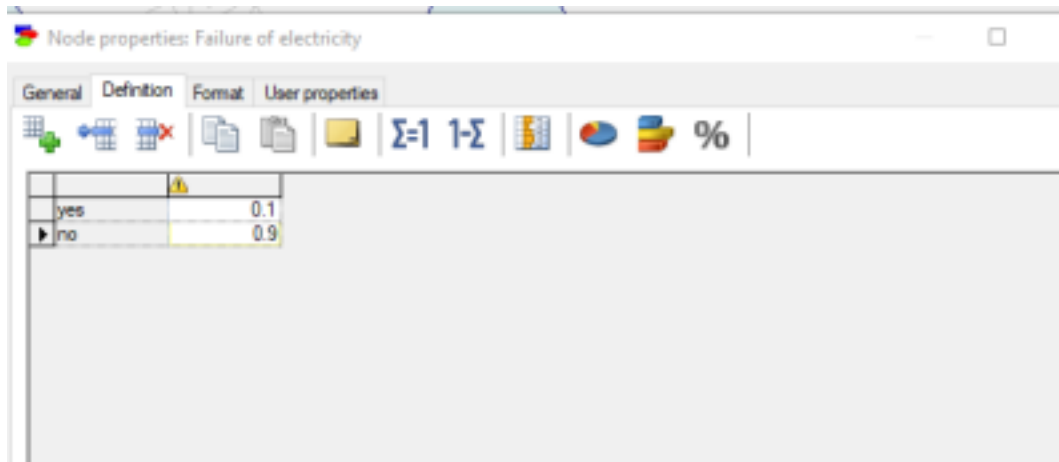
FAST NUCES - Karachi 5

National University of Computer & Emerging Sciences - NUCES -

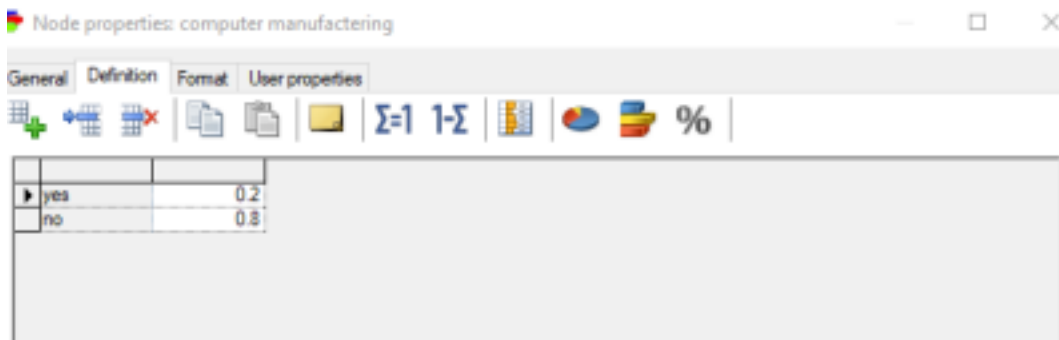


Karachi FAST School of Computing

- To add the tables double click on nodes go to the definition tab.



- We had defined the 2 states for failure of electricity with probabilities of individual events.



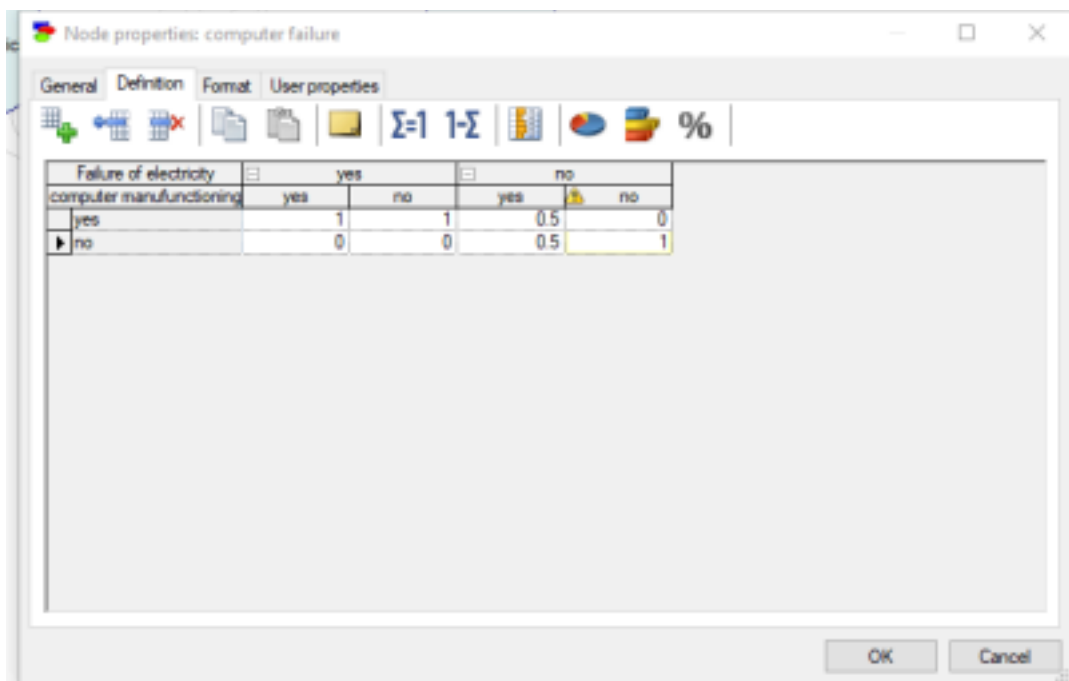
FAST

NUCES - Karachi 6

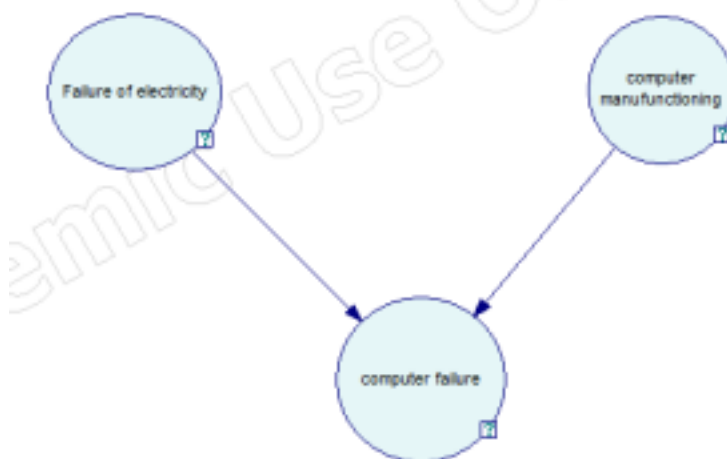


National University of Computer & Emerging Sciences - NUCES - Karachi FAST School of Computing

- Probabilities at second node (computer malfunctioning) . In the same way, define probabilities for the third node as well.



- After defining the probability tables, the arc color will change to blue.



FAST NUCES - Karachi 7

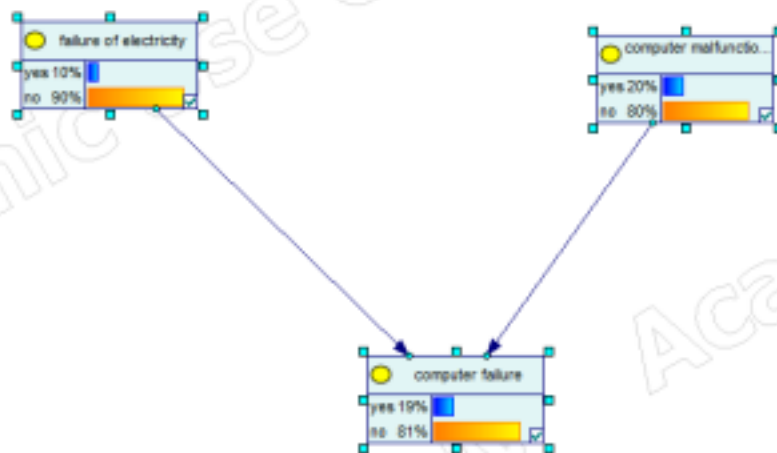
National University of Computer & Emerging Sciences -
NUCES - Karachi

FAST School of Computing

- Now everything is defined, the next step is to run the simulation.



- The question marks have been converted to tick marks once probability tables are properly given.



- We can simulate any probability value and check for results.

FAST NUCES - Karachi 8

National University of Computer & Emerging Sciences -
NUCES - Karachi
FAST School of Computing

4. Burglary Alarm Problem using Bayesian Network

The Burglary Alarm Problem is a standard example used to illustrate Bayesian reasoning. It involves a home alarm system that can be triggered by a burglary or an earthquake. Additionally, two neighbors, John and Mary, may call to report the alarm if they hear it. The Bayesian Network for this problem includes the following **five** Boolean variables:

- Burglary (B) – Whether a burglary has occurred.
- Earthquake (E) – Whether an earthquake has occurred.
- Alarm (A) – Whether the alarm has gone off.
- JohnCalls (J) – Whether John calls to report the alarm.
- MaryCalls (M) – Whether Mary calls to report the alarm.

Network Structure:

The Alarm (A) is directly influenced by Burglary (B) and Earthquake (E). JohnCalls (J) and MaryCalls (M) depend on Alarm (A).

Conditional Probability Tables (CPTs):

Each node in the network has an associated conditional probability table that quantifies the effects of the parent nodes. For example:

- $P(B)$: Probability of a burglary occurring.
- $P(E)$: Probability of an earthquake occurring.
- $P(A | B, E)$: Probability of the alarm going off, given burglary and earthquake.
- $P(J | A)$: Probability of John calling given the alarm.
- $P(M | A)$: Probability of Mary calling given the alarm.

In a home system, the alarm can be triggered by either a burglary or an earthquake. If the alarm goes off, the system notifies two neighbors, John and Mary, who may call the homeowner. Given that both John and Mary have called, what is the probability that a burglary has actually occurred?

```

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Step 1: Define the structure of the Bayesian Network
model = DiscreteBayesianNetwork([
    ('Burglary', 'Alarm'),
    ('Earthquake', 'Alarm'),
    ('Alarm', 'JohnCalls'),
    ('Alarm', 'MaryCalls')
])

# Step 2: Define the CPDs (Conditional Probability Distributions)

# P(Burglary)
cpd_burglary = TabularCPD(variable='Burglary', variable_card=2,
values=[[0.999], [0.001]])

# P(Earthquake)
cpd_earthquake = TabularCPD(variable='Earthquake', variable_card=2,
values=[[0.998], [0.002]])

# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable='Alarm',
    variable_card=2,

```

FAST NUCES - Karachi 10

National University of Computer & Emerging Sciences -
NUCES - Karachi

FAST School of Computing

values=[

```

        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=['Burglary', 'Earthquake'],
    evidence_card=[2, 2]
)

# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable='JohnCalls',
    variable_card=2,
    values=[
        [0.3, 0.9], # JohnCalls = False
        [0.7, 0.1] # JohnCalls = True
    ],
    evidence=['Alarm'],
    evidence_card=[2]
)

# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable='MaryCalls',
    variable_card=2,
    values=[
        [0.2, 0.99], # MaryCalls = False
        [0.8, 0.01] # MaryCalls = True
    ],
    evidence=['Alarm'],
    evidence_card=[2]
)

# Step 3: Add CPDs to the model
model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm, cpd_john, cpd_mary)

# Step 4: Verify the model
assert model.check_model(), "Model is incorrect"

# Step 5: Perform inference
inference = VariableElimination(model)

# Query: What is the probability of a burglary given that both John and Mary
called?
result = inference.query(variables=['Burglary'], evidence={'JohnCalls': 1,
'MaryCalls': 1})
print(result)

```

FAST NUCES - Karachi 11

National University of Computer & Emerging Sciences -
 NUCES - Karachi
 FAST School of Computing

5. Markov Model

Markov models are a type of probabilistic model used to predict the future state of a system, based on its **current state**. The defining feature of a Markov model is that the future state of the system depends solely on the present state and not on the sequence of events that preceded it. This is known as the Markov Property or Memoryless Property. In other words, **Markov models help predict future states using the current (or observed) states of the system, regardless of how the system arrived at its present state.**

A Markov model can be thought of as a finite-state machine where each state has a defined probability of transitioning to other states after one step. These models are ideal for systems that have both hidden (unobservable) and observable states.

Markov models can be classified into two main types:

- **Observable Markov Models:** Where all states and their transitions are known.
- **Hidden Markov Models (HMMs):** Where the system has hidden states that cannot be directly observed, and we must infer them based on observable output.

Markov Model Example: The Marble Bag Problem

To better understand Markov models, let's consider a practical example using a bag of marbles. Suppose the bag contains four marbles:

- 2 Red Marbles
- 2 Blue Marbles

The process works as follows:

1. Select a marble randomly from the bag.
2. Note its color and put it back in the bag.
3. Repeat the process multiple times.

After repeating this process several times, you will notice a pattern: The probability of selecting a red marble is always 2 out of 4, or 50%, and similarly, the probability of selecting a blue marble is 50%.

In this case:

- The past history (i.e., the composition of the bag and the number of marbles) determines the future state (i.e., the probability of selecting a particular color of marble).

FAST NUCES - Karachi 12

National University of Computer & Emerging Sciences -
NUCES - Karachi

FAST School of Computing

- The Markov Property is evident here because the future selection of a marble

(red or blue) only depends on the contents of the bag at the current step (the probability distribution between the red and blue marbles). It does not depend on the order or the sequence of marbles that were previously selected.

Thus, we can model this process using a Markov Chain, where the two states (Red and Blue) transition between each other based on certain probabilities. For this example:

- From a red marble (R), the next state could be either red (R) or blue (B) with equal probabilities (50% each).
- Similarly, from a blue marble (B), the next state could be either red (R) or blue (B) with equal probabilities.

```
import numpy as np

# Define the states and transition matrix
states = ["Red", "Blue"]
transition_matrix = np.array([[0.5, 0.5], # From Red -> Red or Blue
                              [0.5, 0.5]]) # From Blue -> Red or Blue

# Function to simulate the Markov process
def simulate_markov_process(initial_state, num_steps):
    current_state = initial_state
    state_sequence = [current_state]

    for _ in range(num_steps):
        if current_state == "Red":
            next_state = np.random.choice(states,
            p=transition_matrix[0])
        else:
            next_state = np.random.choice(states,
            p=transition_matrix[1])

        state_sequence.append(next_state)
        current_state = next_state

    return state_sequence

# Simulate the process starting from "Red" and for 10 steps
initial_state = "Red"
num_steps = 10
state_sequence = simulate_markov_process(initial_state, num_steps)

# Output the sequence of states
print(f"State sequence for {num_steps} steps starting from {initial_state}:")
print(" -> ".join(state_sequence))
```

FAST NUCES - Karachi 13

National University of Computer & Emerging Sciences -
NUCES - Karachi
FAST School of Computing